REACT NATIVE

Complete Beginner-to-Production Handbook

Setup • Architecture • Navigation • State • APIs • Reusable UI • Native Features • Security • Testing • Deployment

---

Audience: React.js developers beginning React Native, including developers preparing internal team documentation.

Reference stack: Expo, React Native, TypeScript, Expo Router, TanStack Query, Zustand or Redux Toolkit, React Hook Form and Zod.

Revision: October 2026. Exact commands and APIs may change with SDK releases; always follow the linked version-matched documentation.

How to use this handbook

Read chapters 1-7 in order and build the accompanying TaskFlow mini-app as you go. Chapters 8-19 focus on real-world engineering and production. The final sections provide checklists, interview answers, glossary and official references.

1. Mental model: What is React Native?

React Native uses React components and JavaScript/TypeScript to describe interfaces rendered using platform-native UI primitives. You build Android and iOS apps from largely shared business logic; some features still require platform-specific configuration or native modules. Expo is an application framework and tooling ecosystem built around React Native. React Native is not a website wrapped in a WebView by default.

Layer	React Web	React Native
UI	HTML elements like div and button	View, Text, Pressable, TextInput
Rendering	Browser DOM	Platform-native host views
Styles	CSS files/CSS modules	JavaScript style objects, StyleSheet
Navigation	URL/browser router	Native stacks, tabs, deep links
Storage	localStorage, IndexedDB	AsyncStorage, SQLite, secure storage
Network	fetch/Axios	fetch/Axios; device networking
Build	Website bundle	Android APK/AAB, iOS app/IPAs
APIs	Browser window/document	Native device modules; no DOM
Testing	Browser emulators	Physical devices + Android/iOS simulators

Concept mapping

```
// React (web)
<div className="card"><h2>Hello</h2><button onClick={save}>Save</button></div>

// React Native (mobile)
<View style={styles.card}>
  <Text style={styles.title}>Hello</Text>
  <Pressable onPress={save}><Text>Save</Text></Pressable>
</View>
```

- React hooks, props, component composition, TypeScript and many state libraries transfer directly.
- There is no document.querySelector(), CSS DOM, window.localStorage or HTML form by default.
- React Native Flexbox defaults differ: flexDirection defaults to column. Numeric widths and spacing are density-independent points.

2. Development options: Expo vs bare React Native

Choice	Best when	Trade-off
Expo managed workflow	New apps, fast onboarding, common native features, Expo Router	Requires understanding config plugins and development builds
Expo development build	Custom native modules and app-specific native config	Build an app binary when native dependencies change
Bare React Native	Existing native Android/iOS project or specialized native control	More Gradle, Xcode, Pods and upgrade maintenance

Recommendation: For your first app choose the Expo default TypeScript template. Expo Go is convenient for initial experimentation, but it cannot run every third-party custom native module; use a development build when needed. Windows can build/run Android locally, but local iOS simulator and Xcode workflows require macOS; cloud iOS builds may be available through EAS.

3. Install React Native with Expo (Windows / Android first)

- Install the current Node.js LTS, VS Code, Git and optionally Android Studio with Android SDK and emulator.
- Install Expo Go on your Android phone for compatible starter apps; connect computer and phone to the same network or use Expo tunnel if needed.
- Create a new Expo app, inspect generated folder structure, then run the development server.

```
node -v
npm -v
npx create-expo-app@latest taskflow
cd taskflow
npx expo start
# Scan QR with Expo Go; press a to launch Android emulator (if configured).
# If environment/network needs help:
npx expo start --tunnel
```

Important: The current default template typically includes Expo Router. Do not blindly overwrite or mix its src/app and app directories; inspect the generated project. Use npx expo install for Expo-native libraries so package versions align with your SDK. Use npm install for ordinary JavaScript dependencies where appropriate.

Android emulator

- Install Android Studio, SDK platform/platform-tools, emulator and at least one Android Virtual Device (AVD).
- Configure Android SDK PATH/ANDROID_HOME as required by your installed SDK and open an AVD before pressing a in Expo.
- If the emulator cannot reach the API on your computer: Android emulator usually uses 10.0.2.2 for host loopback; a physical phone needs the computer LAN IP, an HTTPS endpoint or a tunnel.

Development build and production builds

```
npm install -g eas-cli   # optional; npx eas-cli also works
eas login
eas build:configure
# Build profile defined in eas.json (set developmentClient true if needed):
eas build --platform android --profile development
# Production distribution example:
eas build --platform android --profile production
```

Build profile output depends on configuration: Android App Bundle (AAB) is normally used for Play Store distribution; APK may be configured for direct installation/internal testing. Signing, app identifier, permissions, privacy details and store accounts must be configured separately. Development builds generally need expo-dev-client installed and an appropriate eas.json profile.

4. Project anatomy and mobile runtime

```
taskflow/
  src/
    app/                  # Route files: index.tsx, _layout.tsx, etc.
    components/           # Reusable visual components
    features/             # Feature-specific screens, hooks and logic
    hooks/                # Shared hooks
    services/             # API clients, integrations
    stores/               # Global client state
    types/                # TypeScript models
    constants/            # Theme, app constants
  assets/                 # Local icons/images/fonts
  app.json                # Expo configuration
  package.json
  tsconfig.json
```

Routing conventions change by template: some projects use app/ at the repository root rather than src/app/. Metro bundles JavaScript, Fast Refresh updates code during development, and native binaries supply platform capabilities. A JS change can refresh without reinstalling the app, but adding certain native modules or modifying native configuration may require a new development build.

5. Core components and styling

Component	Purpose	Typical use
View	Layout container	Card, screen wrapper
Text	Display text	Titles, labels, descriptions
TextInput	Text entry	Search, login forms
Pressable	Touch/press interaction	Custom buttons
ScrollView	Render all scroll children	Small forms/settings
FlatList	Virtualized long list	Feeds, tasks, products
Image	Display images	Avatar/product image
SafeAreaView concept	Avoid notches/insets	Use safe-area-context library
Modal	Overlaid interface	Confirm dialog

```
import { View, Text, Pressable, StyleSheet } from 'react-native';

type Props = { title: string; onPress: () => void };
export function PrimaryButton({ title, onPress }: Props) {
  return (
    <Pressable onPress={onPress} style={styles.button}
      accessibilityRole="button">
      <Text style={styles.label}>{title}</Text>
    </Pressable>
  );
}
const styles = StyleSheet.create({
  button: { backgroundColor: '#1677C7', padding: 14,
            borderRadius: 10, alignItems: 'center' },
  label: { color: '#fff', fontWeight: '700' },
});
```

- Use flexbox, percentages, flex, gap (supported versions), dimensions and Safe Area insets for responsive layout.
- Avoid fixed screen sizes, tiny touch targets, hard-to-read text and assumptions about keyboard/notch spacing.
- Platform.select() or *.android.tsx / *.ios.tsx can handle platform-specific behavior.

6. Navigation: Expo Router in depth

Expo Router is file-based: each screen file in src/app (or app) maps to a navigable route. _layout.tsx configures navigators and shared providers. index.tsx is the index route. (tabs) is a route group that does not add to URL; [id].tsx is a dynamic route. A Stack pushes detail screens, Tabs expose peer destinations, a Drawer supplies a side menu, and modal presentation overlays a screen.

```
src/app/
  _layout.tsx             # Root navigator
  index.tsx               # /
  login.tsx               # /login
  (tabs)/
    _layout.tsx           # Tabs config
    index.tsx             # Main tab
    settings.tsx          # Settings tab
  task/
    [id].tsx              # /task/123
```

```
// src/app/_layout.tsx
import { Stack } from 'expo-router';
export default function RootLayout() {
  return <Stack><Stack.Screen name="(tabs)" options={{
    headerShown: false }} /></Stack>;
}

// src/app/(tabs)/_layout.tsx
import { Tabs } from 'expo-router';
export default function TabsLayout() {
  return <Tabs><Tabs.Screen name="index" options={{ title: 'Tasks' }} />
    <Tabs.Screen name="settings" options={{ title: 'Settings' }} />
  </Tabs>;
}
```

```
// Any screen/component
import { router, Link } from 'expo-router';
router.push({ pathname: '/task/[id]', params: { id: '123' } });
router.replace('/login');  // replace history (e.g., after sign-out)
router.back();
// <Link href="/settings">Settings</Link> if route resolves to /settings

// src/app/task/[id].tsx
import { useLocalSearchParams } from 'expo-router';
const { id } = useLocalSearchParams<{ id: string }>();
```

Production auth requires protected routes or navigation guards backed by validated session state, not merely hiding a tab. Deep links can open routes directly. On latest Expo Router versions, APIs differ from standalone React Navigation: follow the documentation that matches your installed Expo SDK.

React Navigation alternative

React Navigation is a code-configured navigation ecosystem commonly used in non-Expo-Router projects. It provides stacks, tabs and drawers. Do not mix independent routing stacks without a clear architecture. Teams with an existing React Navigation app usually keep it rather than migrate simply for fashion.

7. Local, global, server and persistent state

Local UI state belongs to one component (open modal, current input). Shared global client state is used across several screens (theme, cart filters, unsaved draft). Server state is fetched remotely (tasks, profile, order history), and needs caching, freshness, refetch and invalidation. Persistent state survives app restart (non-sensitive preferences, secure auth session). The categories can overlap but should not be collapsed into one enormous store.

Problem	First choice	Notes
Input/modal/toggle	useState	Keep closest to consumer

Problem	First choice	Notes
Complex local transitions	useReducer	Explicit action handling
Rare shared simple state	Context API	Avoid frequent large-context updates
Small global client state	Zustand	Simple selectors/actions
Large predictable global updates	Redux Toolkit	Slices, DevTools, middleware
Remote API response/cache	TanStack Query	query keys and invalidation
Persistent ordinary preference	AsyncStorage / SQLite	Not secret storage
Access/refresh token	SecureStore / Keychain / Keystore	Follow threat model and auth protocol

```
// Local useState example
const [search, setSearch] = useState('');
const [isOpen, setIsOpen] = useState(false);

// useReducer example
function reducer(state: number, action: { type: 'inc' | 'dec' }) {
  if (action.type === 'inc') return state + 1;
  return state - 1;
}
const [count, dispatch] = useReducer(reducer, 0);
```

Context API: simple shared values

```
import { createContext, useContext, useState } from 'react';
const ThemeContext = createContext<{
  dark: boolean; toggle: () => void
} | null>(null);
export function ThemeProvider({ children }:
  { children: React.ReactNode }) {
  const [dark, setDark] = useState(false);
  return <ThemeContext.Provider value={{ dark,
    toggle: () => setDark(v => !v) }}>{children}</ThemeContext.Provider>;
}
export function useThemeMode() {
  const value = useContext(ThemeContext);
  if (!value) throw new Error('Missing ThemeProvider');
  return value;
}
```

Zustand: shared client-state example

```
npm install zustand

import { create } from 'zustand';
type CartStore = {
  items: string[];
  add: (id: string) => void;
  clear: () => void;
};
export const useCart = create<CartStore>((set) => ({
  items: [],
  add: (id) => set((s) => ({ items: [...s.items, id] })),
  clear: () => set({ items: [] }),
}));
// In component: const items = useCart(s => s.items);
```

Redux Toolkit: predictable application-level state

```
npm install @reduxjs/toolkit react-redux

import { createSlice, configureStore } from '@reduxjs/toolkit';
import { Provider, useDispatch, useSelector } from 'react-redux';
const cartSlice = createSlice({
  name: 'cart', initialState: { count: 0 },
  reducers: { add(state) { state.count += 1; },
              clear(state) { state.count = 0; } },
});
export const { add, clear } = cartSlice.actions;
export const store = configureStore({
  reducer: { cart: cartSlice.reducer },
});
export type RootState = ReturnType<typeof store.getState>;
// Wrap root tree: <Provider store={store}>...</Provider>
// In screen: useSelector((s: RootState) => s.cart.count)
```

Redux Toolkit adds patterns and debugging that can be valuable in bigger teams. Use selectors to limit rerenders, do not put every keystroke into Redux, and avoid copying all query responses into Redux when a server-state library already owns them.

8. API calls with fetch and Axios

React Native can call HTTP/HTTPS APIs through fetch or Axios. The backend remains separate: your phone sends requests to an API, the API authenticates and authorizes, executes business logic and uses the database. Never connect directly from the app to a production database with embedded credentials.

```
type Task = { id: string; title: string; done: boolean };
const API_URL = process.env.EXPO_PUBLIC_API_URL;
export async function getTasks(signal?: AbortSignal): Promise<Task[]> {
  if (!API_URL) throw new Error('API URL is not configured');
  const response = await fetch(`${API_URL}/tasks`, { signal });
  if (!response.ok) throw new Error(`HTTP ${response.status}`);
  return response.json() as Promise<Task[]>;
}
// GET /tasks, POST /tasks, PATCH /tasks/:id, DELETE /tasks/:id
```

POST with Axios and structured errors

```
npm install axios

import axios from 'axios';
export const api = axios.create({
  baseURL: process.env.EXPO_PUBLIC_API_URL,
  timeout: 10000,
});
export async function createTask(title: string) {
  const { data } = await api.post('/tasks', { title });
  return data as { id: string; title: string; done: boolean };
}
try { await createTask('Learn React Native'); }
catch (error) {
  if (axios.isAxiosError(error)) {
    console.log(error.response?.status ?? 'Network error');
  }
}
```

- Handle loading, empty, error, success and offline states rather than rendering just response data.
- Use AbortController (or request cancellation) when requests become stale and avoid race conditions in search.
- Use pagination/infinite scrolling for long feeds; validate server responses when correctness matters.
- Only retry transient/idempotent requests automatically; use capped exponential backoff and honor Retry-After where possible.

Development API hostnames and environment

localhost inside a simulator/phone usually points to that device, not your laptop. Android emulator can often use http://10.0.2.2:PORT for a backend running on the host; real devices use your laptop LAN IP such as http://192.168.x.x:PORT if reachable. iOS simulator may access host localhost in common setups. Prefer HTTPS for deployments; cleartext Android policies, firewall and LAN setup can affect local testing.

Expo EXPO_PUBLIC_* variables are embedded in the app bundle; they are public. Never put API secrets, database passwords, private keys or admin tokens into these variables. Use a server-side backend for secrets.

9. TanStack Query: correct server-state handling

TanStack Query manages fetching, cached results, stale times, deduplication, invalidation, retries and mutations. It does not replace local client UI state. QueryProvider must wrap any screen calling useQuery().

```
npm install @tanstack/react-query

// src/app/_layout.tsx - simplified provider illustration
import { QueryClient, QueryClientProvider } from '@tanstack/react-query';
import { Stack } from 'expo-router';
import { useState } from 'react';
export default function RootLayout() {
  const [client] = useState(() => new QueryClient());
  return <QueryClientProvider client={client}>
    <Stack />
  </QueryClientProvider>;
}

// features/tasks/useTasks.ts
import { useQuery, useMutation, useQueryClient } from '@tanstack/react-query';
export const useTasks = () => useQuery({
  queryKey: ['tasks'], queryFn: () => getTasks(), staleTime: 30000,
});
export const useAddTask = () => {
  const client = useQueryClient();
  return useMutation({ mutationFn: createTask,
    onSuccess: () => client.invalidateQueries({ queryKey: ['tasks'] }) });
};
```

In mobile apps, configure app focus and network connectivity handling when needed (React Native AppState and connectivity listeners); browsers and React Native do not have identical focus/online events. For offline persistence use compatible persisters carefully and avoid persisting sensitive data unencrypted.

10. Reusable UI components and a design system

Use components for repeated behavior and visuals, hooks for reusable logic, services for data access, and features for domain separation. Prefer composition and typed props instead of a giant component with many conditional branches.

```
// src/components/AppTextField.tsx
import { TextInput, View, Text, StyleSheet } from 'react-native';
type Props = {
  label: string; value: string; onChangeText: (s: string) => void;
  error?: string; secureTextEntry?: boolean;
};
export function AppTextField({ label, error, ...input }: Props) {
  return <View style={{ marginBottom: 12 }}>
    <Text style={styles.label}>{label}</Text>
    <TextInput {...input} style={styles.input}
      accessibilityLabel={label} />
    {!!error && <Text style={styles.error}>{error}</Text>}
  </View>;
}
const styles = StyleSheet.create({
  label: { fontWeight: '600', marginBottom: 6 },
  input: { borderWidth: 1, borderColor: '#94A3B8',
           padding: 12, borderRadius: 8 },
  error: { color: '#B42318' },
});
```

- Build shared Button, Input, Screen, LoadingState, EmptyState, ErrorState, TaskCard, Avatar and ConfirmDialog components.
- Maintain spacing scale, typography, colors, radius, light/dark palettes, status colors and accessibility requirements.
- For FlatList, use stable keyExtractor; avoid nesting long FlatLists inside ScrollView.

11. Forms and validation

For simple forms, TextInput controlled state is enough. For bigger forms, React Hook Form + Zod reduces repetitive state logic, supports schema validation, and allows controlled integrations via Controller.

```
npm install react-hook-form zod @hookform/resolvers

import { Controller, useForm } from 'react-hook-form';
import { zodResolver } from '@hookform/resolvers/zod';
import { z } from 'zod';
import { TextInput, Button, Text } from 'react-native';
const schema = z.object({ email: z.email(),
  password: z.string().min(8) });
type Form = z.infer<typeof schema>;
const { control, handleSubmit } = useForm<Form>({
  resolver: zodResolver(schema),
  defaultValues: { email: '', password: '' },
});
// In component render:
<Controller control={control} name="email" render={({ field,
  fieldState }) => <>
  <TextInput value={field.value} onChangeText={field.onChange}
    onBlur={field.onBlur} autoCapitalize="none" />
  {fieldState.error && <Text>{fieldState.error.message}</Text>}
</>} />
<Button title="Submit" onPress={handleSubmit(console.log)} />
```

Adjust Zod email syntax to the installed version if necessary; use z.string().email() in older Zod releases. Always validate again on the backend. Avoid logging passwords and personal data.

12. Authentication and authorization architecture

Typical flow: login screen -> POST /auth/login -> server verifies credentials -> token/session issued -> securely stored as appropriate -> user profile fetched -> protected screens shown -> logout/revoke/clear. Prefer a trusted OIDC/OAuth flow with PKCE when supported by your backend. Do not build ad hoc OAuth flows in WebViews or ship client secrets.

```
// Example storage boundary for a sensitive session token
npx expo install expo-secure-store

import * as SecureStore from 'expo-secure-store';
const TOKEN_KEY = 'access_token';
export async function saveToken(token: string) {
  await SecureStore.setItemAsync(TOKEN_KEY, token);
}
export async function getToken() {
  return SecureStore.getItemAsync(TOKEN_KEY);
}
export async function clearToken() {
  await SecureStore.deleteItemAsync(TOKEN_KEY);
}
```

- SecureStore is more appropriate for small credentials than AsyncStorage, but device compromise and token theft are still possible.
- Plan short-lived access tokens, refresh rotation/revocation, expiry handling, logout, biometric policy and concurrent refresh control.
- Client route guards are UX; every privileged API endpoint must enforce permissions on the server.
- Never assume the existence of a token proves a valid session; verify session with the backend when needed.

13. Common third-party libraries: what, why, when

Category	Typical choices	When to use
Routing	expo-router; React Navigation	Stacks, tabs, deep links
Global state	Zustand; Redux Toolkit; Context	Shared client-side state
Server state	TanStack Query; RTK Query	Cache, mutations, refetch
HTTP	fetch; axios	API layer
Forms	React Hook Form; Zod	Validation and controlled forms
UI systems	React Native Paper; Tamagui; NativeWind	Design system; evaluate SDK compatibility
Icons	@expo/vector-icons; lucide-react-native	Consistent iconography
Storage	AsyncStorage; expo-sqlite; expo-secure-store	Preferences, local DB, secrets
Lists	FlatList; FlashList	Large lists, after measuring performance
Animations	React Native Reanimated; Moti	Smooth/advanced animations
Gestures	react-native-gesture-handler	Gestures, swipes
Images	expo-image; expo-image-picker	Image display and selection
Camera	expo-camera	Photos/barcodes
Location	expo-location	Location after permissions
Notifications	expo-notifications	Local/push messaging
Auth	expo-auth-session; provider SDK	OAuth/OIDC integrations
Testing	Jest; React Native Testing Library; Maestro	Logic/UI/end-to-end
Monitoring	Sentry	Crash/error traces
Connectivity	@react-native-community/netinfo	Network status

Library support is version-dependent and native modules may require config plugins and a development build. Check package release activity, maintenance, platform support, privacy implications and compatibility with your Expo SDK before installation. Do not install everything at once.

14. Phone-specific features and permissions

- Camera/image picker: request permission only when needed, handle denial and cancellation, compress large uploads.
- Location: request foreground access first; background permission requires stronger justification and platform configuration.

- Push notifications: request permission, register device push token, associate it with authenticated user on backend; handle foreground/background tap navigation and token updates.
- Files/share: use platform document picker or sharing modules; never assume desktop file paths or permanent picker URIs.
- Deep linking: map app-specific schemes and universal/app links to routes; validate incoming params as untrusted input.
- Keyboard: avoid inputs hidden behind mobile keyboard; test forms with keyboard open on small devices.

15. Offline-first and local persistence

AsyncStorage is asynchronous key-value persistence appropriate for non-secret preferences. SQLite is for structured local relational data. SecureStore/OS credential stores are designed for small sensitive values. Offline write queues and conflict resolution add complexity and should be explicit product choices.

```
npx expo install @react-native-async-storage/async-storage

import AsyncStorage from '@react-native-async-storage/async-storage';
await AsyncStorage.setItem('theme', 'dark');
const theme = await AsyncStorage.getItem('theme');
await AsyncStorage.removeItem('theme');
```

An offline sync design needs local IDs, a pending-operation queue, ordering/retry policies, deduplication or idempotency keys, conflict resolution (server wins / client wins / merge), and observability. Never claim offline correctness simply because data is cached.

16. Mobile performance and optimization

- Use FlatList for hundreds/thousands of items; use windowing, keyExtractor and pagination. Measure before selecting a different list implementation.
- Minimize unnecessary rerenders with good state boundaries and memoization only where profiling supports it.
- Keep expensive computations out of render paths; avoid repeatedly parsing giant JSON payloads.
- Serve correct image sizes; cache or optimize large remote images; use thumbnails.
- Animations running on UI thread can improve smoothness, but test on lower-end Android devices.
- Use React DevTools / RN DevTools, production-like builds, native profiling and device metrics; debug builds can misrepresent speed.
- Monitor startup time, frame drops, memory, crashes, network usage, battery and bundle size where supported.

17. Security, privacy and reliability

Risk	Recommended practice
Leaked secrets	No private keys in bundle, EXPO_PUBLIC_* or committed .env; keep secrets on backend
Broken access control	Validate each request server-side; verify resource ownership
Token theft	Secure storage, expiry, refresh rotation, revoke sessions
Untrusted links/inputs	Validate schemes/hosts/params and user input
Insecure networking	TLS/HTTPS; restrict debug cleartext exceptions
Sensitive logs	No tokens/PII/passwords in logs, crash metadata or analytics
Dependency risks	Pin versions, audit and upgrade native modules
Permissions abuse	Least permission; clear purpose and privacy notice
Offline data exposure	Threat-model cached PII; encrypt where necessary

Mobile apps are untrusted clients. Obfuscation and hiding buttons are never sufficient authorization. UI protections, certificates and storage methods support security but do not replace correct backend policies.

18. Testing, debugging and accessibility

Testing pyramid: unit-test pure helpers, component-test user interactions and errors, integration-test screens with mocked APIs, and end-to-end test primary flows on devices. Automate a smoke test for login, task list, create/edit, offline state and logout.

```
// Example component test (illustrative)
import { render, fireEvent } from '@testing-library/react-native';
import { PrimaryButton } from '../PrimaryButton';
it('calls action on press', () => {
  const onPress = jest.fn();
  const screen = render(<PrimaryButton title="Save" onPress={onPress} />);
  fireEvent.press(screen.getByText('Save'));
  expect(onPress).toHaveBeenCalledTimes(1);
});
```

- Accessibility: proper roles and labels, large touch targets, screen reader navigation, dynamic font sizes, color contrast and text truncation.
- Common debugging: inspect Metro output, Android Logcat, iOS console, red screen stacks, connectivity and permissions.
- Test Android back behavior, small devices, tablets, keyboard, rotation, slow networks and denied permissions.

19. Release process: Android and iOS

- Set app identifiers, version/build numbers, name, icons, splash assets and correct runtime permissions.
- Create development/internal testing builds and test them on real devices, not only Expo Go.
- Configure signing credentials, EAS build profiles, environment and production API endpoints.

- Prepare Play Store listing, privacy disclosures, data safety information, screenshots and test/release tracks.
- For iOS distribution, arrange Apple developer requirements, signing, provisioning and App Store Connect metadata.
- Roll out gradually, track crashes and API errors; plan rollback/remediation. OTA JS updates are subject to native runtime compatibility and store policies.

20. Practical mini-project: TaskFlow

Build TaskFlow: a mobile task manager with login, task list, add/edit/delete, task detail, tabs, filters and offline handling. Use an authenticated REST backend or a mock service first. The reference architecture separates view logic, client state and server state.

```
src/
  app/_layout.tsx
  app/index.tsx                # auth/loading redirect
  app/login.tsx
  app/(tabs)/_layout.tsx
  app/(tabs)/index.tsx         # tasks
  app/(tabs)/settings.tsx
  app/task/[id].tsx
  components/PrimaryButton.tsx
  components/AppTextField.tsx
  components/TaskCard.tsx
  features/tasks/api.ts
  features/tasks/queries.ts
  features/auth/session.ts
  stores/usePreferences.ts
  services/api.ts
  types/task.ts
```

Feature architecture flow

```
Screen -> typed components -> feature hook -> TanStack Query
                                                |
                                    api service -> HTTPS backend
Screen -> Zustand/Context -> user preferences and temporary UI state
SecureStore -> session credential; backend -> permission enforcement
```

Build the task listing screen

```
// src/app/(tabs)/index.tsx
import { ActivityIndicator, FlatList, Text, View } from 'react-native';
import { useTasks } from '../../features/tasks/queries';
export default function TasksScreen() {
  const { data = [], isPending, isError, refetch } = useTasks();
  if (isPending) return <ActivityIndicator />;
  if (isError) return <Text onPress={() => refetch()}>Retry loading</Text>;
  return <FlatList data={data} keyExtractor={(task) => task.id}
    ListEmptyComponent={<Text>No tasks yet</Text>}
    renderItem={({ item }) => <View style={{ padding: 16 }}>
      <Text>{item.title}</Text>
    </View>} />;
}
```

In an actual app ensure types/files/imports match your project. The task screen assumes its root QueryClientProvider and task API are already configured. Add optimistic updates only after basic invalidation behavior works; implement rollback and retry/error feedback.

Milestones and acceptance criteria

Stage	Deliverable	Definition of done
Day 1-2	Expo app + layout	Launch on emulator and phone, basic UI
Day 3-4	Router/screens	Tabs, detail route and back navigation
Day 5-6	API + Query	Loading/error/empty state, list and create
Day 7-8	Forms/global state	Validated form + stored preference
Day 9-10	Authentication	Session restore, logout, backend protected API
Day 11-12	Testing/release	Smoke tests, production-like Android build

21. Common beginner mistakes and fixes

Mistake	Better approach
Using div, h1, onClick	Use View, Text, onPress and RN styles
API localhost on real phone	Use reachable LAN IP / HTTPS backend
Installing any random native package	Verify Expo version + dev-build requirement
Keeping access token in AsyncStorage	Use secure storage + safe token lifecycle
Saving all API data in Redux	Prefer query cache for server state
Putting reusable components in app/	Place them outside route directory
Rendering 2000 cards with ScrollView	Use FlatList/pagination
Ignoring status/keyboard areas	Safe area + keyboard-aware UI design
Only testing Expo Go	Test actual Android/iOS builds
Reading old blog for install steps	Check version-specific official SDK documentation

22. React versus React Native interview Q&A

What is React Native?

A framework for building native mobile UIs with React; JS/TS code orchestrates native host views and native modules.

Is React Native the same as React?

They share component and hook concepts; React web targets DOM while React Native targets native platform primitives.

What is Expo?

A React Native application framework/toolchain for project setup, native modules, routing, builds and updates.

Why FlatList over ScrollView?

FlatList virtualizes large lists rather than mounting every row at once.

What is the difference between local/global/server state?

Local is component-specific; global is shared client state; server state is remote data with cache/freshness needs.

When use Redux Toolkit vs Zustand?

Redux when team conventions, debugging and structured transitions matter; Zustand for leaner shared state.

What does TanStack Query solve?

Caching, stale data, fetching, mutations, retries and query invalidation.

What is Expo Router?

Routes defined by files and layouts, enabling stack/tab navigation and deep links.

Where store tokens?

In platform-backed secure storage with server-side token validation, expiration, revocation and renewal.

How is navigation guarded?

Determine authenticated session, conditionally expose protected routes and independently enforce API authorization.

Why does API localhost fail on phone?

localhost identifies the mobile device; the backend on laptop needs a reachable host address.

Do you need Java/Kotlin/Swift?

Not for most shared app UI; native expertise is needed for complex native integrations and platform debugging.

23. Recommended learning sequence

- Week 1 - Install Expo, learn core RN primitives, Flexbox, inputs, FlatList and device testing.
- Week 2 - Expo Router stack/tabs, props, component composition, local state, reusable components.
- Week 3 - REST APIs, Axios/fetch, Query, pagination, retries, typing and error/empty states.
- Week 4 - Authentication, secure token lifecycle, forms, native permissions, persistence and basic testing.
- Week 5 - Optimization, accessibility, offline support, debug builds and release pipeline.

24. Documentation and library reference links

React Native docs - https://reactnative.dev/docs/getting-started

React Native environment setup - https://reactnative.dev/docs/set-up-your-environment

Expo documentation - https://docs.expo.dev/

Expo create project - https://docs.expo.dev/get-started/create-a-project/

Expo Router introduction - https://docs.expo.dev/router/introduction/

Expo Router core concepts - https://docs.expo.dev/router/basics/core-concepts/

Expo navigation guide - https://docs.expo.dev/develop/app-navigation/

Expo EAS Build - https://docs.expo.dev/build/introduction/

React Navigation - https://reactnavigation.org/docs/getting-started

Redux Toolkit - https://redux-toolkit.js.org/

Zustand - https://zustand.docs.pmnd.rs/

TanStack Query - https://tanstack.com/query/latest/docs/framework/react/overview

React Hook Form - https://react-hook-form.com/

React Native Testing Library - https://callstack.github.io/react-native-testing-library/

Expo SecureStore - https://docs.expo.dev/versions/latest/sdk/securestore/

Expo AsyncStorage - https://docs.expo.dev/versions/latest/sdk/async-storage/

25. Manager handover checklist

- Confirm whether the team has an existing bare React Native project or a new Expo application.
- Document exact SDK/React Native/Node versions and lockfile; use version-matched commands.
- Identify existing routing choice, authentication protocol, API base URLs and environment scheme.
- Agree on local versus global versus server-state ownership and a folder structure.
- Create working sample of login, list fetch, create mutation, protected navigation and a reusable form.
- Document Android/iOS build/signing ownership, credentials management, QA devices and release process.
- Add accessibility, logging/privacy, crash reporting, offline assumptions and security review to definition of done.

Summary: React Native combines familiar React logic with mobile-specific navigation, native modules, platform builds and device lifecycle. Learn the primitives first; adopt Expo Router, a query-cache approach and a small maintainable design system; add global state and native libraries only for clear requirements.